HW1: Lean-Into-Corner Controller & Vehicle Sensor Data Analysis

Gain Scheduling Baseline + ML Gain Scheduler Comparison — Smartphone Accel/GPS Analysis

Kshitij Gupta

Feb 9, 2026

Overview (What this report covers)

This report answers **Homework 1**:

- **Problem 1:** Lean-Into-Corner Controller (gain scheduling baseline + ML gain scheduler comparison).
- **Problem 2:** Smartphone-based acceleration/GPS measurement, video synchronization, and time-scale/actuator analysis.

1 Problem 1: Lean-Into-Corner Controller

2 Problem Setup and Modeling Assumptions

2.1 Goal: comfort-first lean

When a vehicle turns with lateral acceleration a_y, passengers feel a lateral inertial effect unless the cabin/vehicle leans. A comfort-first approach chooses a desired roll angle ϕ_{des} such that gravity provides a component that cancels the turning inertial effect (in the vehicle body frame).

2.2 Roll (lean) dynamics model

We use a single-DOF roll model with linear stiffness and damping:

$$I_\phi \ddot{\phi} + c_\phi \dot{\phi} + k_\phi \phi = \tau_{\text{act}} \tag{1}$$

where:

- ϕ is roll angle (rad), $\dot{\phi}$ is roll rate (rad/s)
- I_ϕ is roll inertia
- k_ϕ is roll stiffness and c_ϕ is roll damping
- τ_{act} is the actuator roll torque

Parameter values (used in simulation).

Parameter	Value	Units
I_ϕ	600	kg·m^2
k_ϕ	10000	N·m/rad
c_ϕ	2000	N·m·s/rad
g	9.81	m/s^2

2.3 Modeling and Control Assumptions

The following assumptions are made in this work:

- The vehicle roll dynamics are approximated as a single-degree-of-freedom linear system.
- Roll stiffness and damping are constant and known.
- Actuator torque is applied directly to the roll DOF (no actuator dynamics or delays).
- Vehicle speed and path curvature are known and smooth.
- The desired lean angle is computed assuming quasi-static balance between gravity and lateral acceleration.
- Passenger felt lateral acceleration is approximated by a reduced model proportional to roll tracking error.
- Tire dynamics, load transfer, and lateral slip are not explicitly modeled.
- For the ML scheduler, training is performed offline and the learned gains are bounded for safety.

3 Turn Profile and Desired Lean Angle

3.1 Curvature and lateral acceleration

For a path curvature κ (1/m) and speed v (m/s), lateral acceleration is:

$$a_y = v^2 \kappa \tag{2}$$

We use a smooth curvature ramp-in/out profile to avoid actuator jerk and improve passenger comfort.

3.2 Smooth ramp-in / ramp-out curvature

We define a scalar ramp $s(t) \in [0, 1]$ using a C^1 smoothstep:

$$\text{smoothstep}(x) = 3x^2 - 2x^3, \quad x \in [0, 1] \tag{3}$$

Then for a signed turn radius R (m), curvature is:

$$\kappa(t) = \frac{s(t)}{R} \tag{4}$$

In our runs, the turn begins at $t = 1\,\text{s}$, ramps in for $1\,\text{s}$, holds, then ramps out so that the turn ends at $t = 10\,\text{s}$ (i.e., curvature returns to zero by $t = 10$).

3.3 Comfort-based desired lean angle

The comfort-first desired roll angle is:

$$\phi_{\text{des}} = -\arctan\left(\frac{a_y}{g}\right) \tag{5}$$

This is clamped to a safety limit $|\phi_{\text{des}}| \leq \phi_{\max}$, where $\phi_{\max} = 10\,\text{deg}$.

4 Controller Design (Comfort-First)

4.1 High-level structure

We use a two-loop structure:

- Outer loop: roll angle tracking $\rightarrow$ produces roll-rate command $\dot{\phi}_{\text{cmd}}$ via PI.
- Inner loop: roll-rate servo $\rightarrow$ produces feedback torque τ_{fb}.
- Feedforward torque: compensates roll spring torque at the desired lean.

4.2 Outer PI: angle error to rate command

Let $e = \phi_{\text{des}} - \phi$ and $e_I = \int e\,dt$. The outer loop computes:

$$\dot{\phi}_{\text{cmd}} = K_p e + K_i e_I \tag{6}$$

A comfort-based saturation limits the commanded rate:

$$|\dot{\phi}_{\text{cmd}}| \leq \dot{\phi}_{\text{cmd,max}}$$

and conditional integration (anti-windup) is used so the integrator does not grow when saturated.

4.3 Inner rate servo: rate error to feedback torque

The inner loop produces a damping-like torque:

$$\tau_{\text{fb}} = K_{\text{rate}}\left(\dot{\phi}_{\text{cmd}} - \dot{\phi}\right) \tag{7}$$

4.4 Feedforward torque (steady-state holding torque)

At steady state (constant $\phi \approx \phi_{\text{des}}$, $\dot{\phi} \approx 0$), the roll spring produces a restoring torque $k_\phi \phi$. To hold the desired lean without relying on feedback, we add:

$$\tau_{\text{ff}} = k_\phi \phi_{\text{des}} \tag{8}$$

Physical interpretation: this is the actuator torque required to balance the suspension/roll stiffness at the leaned equilibrium. It does *not* directly "cancel lateral acceleration"; rather, the *lean angle* changes the direction of gravity in the body frame, which reduces the lateral acceleration felt by the passenger.

4.5 Total actuator torque

$$\tau_{\text{act}} = \tau_{\text{ff}} + \tau_{\text{fb}} \tag{9}$$

5 Block Diagram and Free-Body Diagram (FBD)

5.1 Control system block diagram

Figure 1 shows the controller structure used in both the gain-scheduled baseline and the ML-based scheduler (only the mapping to gains differs).

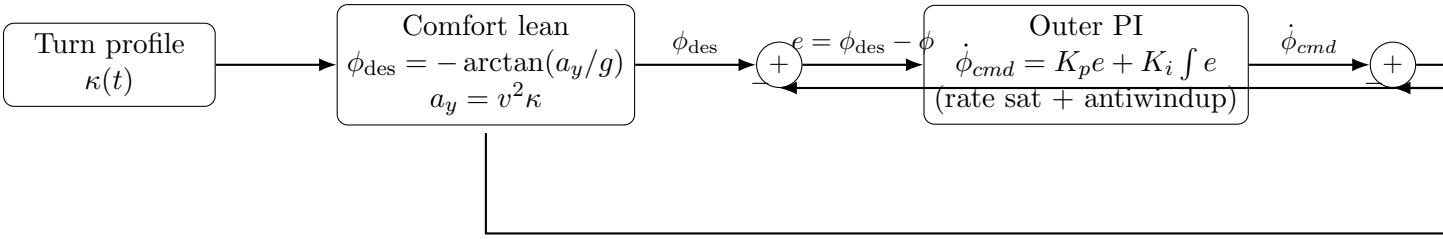
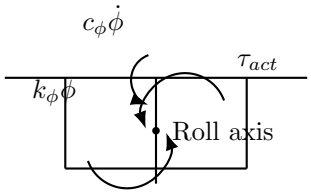


Figure 1: Block diagram of the comfort-first lean controller. Gain scheduling (table-based) or ML-based scheduling changes how (K_p, K_{rate}) are selected.

5.2 Roll-axis free-body diagram (torque balance)

A simplified torque balance about the roll axis is shown in Figure 2. The actuator torque must overcome the roll spring and damper to achieve the desired lean motion.



Dynamics: $I_\phi\ddot{\phi} = \tau_{act} - c_\phi\dot{\phi} - k_\phi\phi$

Figure 2: Simplified roll-axis torque balance (FBD). The feedforward term $\tau_{ff} = k_\phi\phi_{des}$ compensates the spring torque at the desired lean.

6 (a) Gain Scheduling Algorithm (Baseline)

6.1 Why gain scheduling?

As speed v and curvature magnitude $|\kappa|$ increase, the desired lean changes faster and the controller must respond more strongly to maintain comfort. Fixed gains can work for a single operating point but may be too sluggish or too aggressive elsewhere. Therefore, we schedule K_p and K_{rate} as smooth functions of $(v, |\kappa|)$ while keeping K_i constant.

6.2 2D scheduling with bilinear interpolation

We define grids:

$$v \in \{5, 10, 20\}\,\text{m/s}, \quad |\kappa| \in \left\{0, \frac{1}{400}, \frac{1}{200}, \frac{1}{100}\right\}\,\text{m}^{-1}$$

with lookup tables $K_p(v, |\kappa|)$ and $K_{rate}(v, |\kappa|)$. For intermediate values, we use bilinear interpolation to avoid jumps.

6.3 Pseudocode for scheduling + control loop

Listing 1: High-level pseudocode for the gain-scheduled controller.

```
for each time step t:
    kappa(t) = curvature_profile(t)          # smooth ramp in/out
    ay = v^2 * kappa(t)
    phi_des = -atan(ay/g)                    # comfort lean

    # schedule gains smoothly using bilinear interpolation
    Kp    = bilinear(v, abs(kappa), v_grid, kappa_grid, Kp_table)
    Krate = bilinear(v, abs(kappa), v_grid, kappa_grid, Krate_table)

    # outer PI -> roll rate command (with saturation + anti-windup)
    e = phi_des - phi
    phi_dot_cmd = sat( Kp*e + Ki*e_int, +/-phi_dot_cmd_max )
    if not saturated: e_int += e*dt

    # feedforward + feedback torque
    tau_ff = kphi * phi_des
    tau_fb = Krate * (phi_dot_cmd - phi_dot)
    tau_act = tau_ff + tau_fb

    # plant integration
    phi_ddot = (tau_act - cphi*phi_dot - kphi*phi) / Iphi
    phi_dot += phi_ddot * dt
    phi     += phi_dot * dt

    # comfort metric
    a_lat_feel = g*(phi_des - phi)
```

6.4 Comfort metric

We use the reduced-model comfort metric:

$$a_{\text{lat,feel}} \approx g\,(\phi_{\text{des}} - \phi) \tag{10}$$

This is zero when $\phi = \phi_{\text{des}}$. During transients, tracking error causes non-zero felt lateral acceleration.

7 (b) Performance Profiling

7.1 What dominates runtime?

For a time step size $\Delta t = 0.001\,\text{s}$ over $T = 15\,\text{s}$, each simulation run executes $\sim 15{,}000$ iterations. The dominant cost per step is:

- computing $\arctan(\cdot)$ for ϕ_{des}

- interpolation for scheduled gains (a few comparisons + lerps)
- scalar plant update (constant-time arithmetic)

7.2 Expected complexity

Each scenario is $O(N)$ where $N = T/\Delta t$. For multi-scenario evaluation (e.g., ML training), runtime scales linearly with number of scenarios.

7.3 Practical profiling (Python)

In Python, a typical approach is to use `time.perf_counter()` around the simulation function or `cProfile`. Common optimizations:

- reduce arctan calls via vectorization (optional)
- precompute curvature profile $s(t)$
- replace Python loops with NumPy vector operations (harder here due to recursion in dynamics)

In real-time deployment, gain scheduling is extremely lightweight; the ML approach also becomes lightweight after training (only a small NN forward pass).

8 Results: Gain-Scheduled Baseline (Provided Plots)

This section includes the plots generated by the gain-scheduled baseline notebook. In Overleaf, place the images in a folder (e.g., `figures/`) and update the paths below.

Note on why gains can look piecewise-constant. In this experiment, v is constant and $|\kappa|$ is nearly constant during the hold phase of the turn. Since the scheduler depends on $(v, |\kappa|)$, $K_p(t)$ and $K_{rate}(t)$ appear nearly constant during steady turning and vary mainly during ramp-in/out.

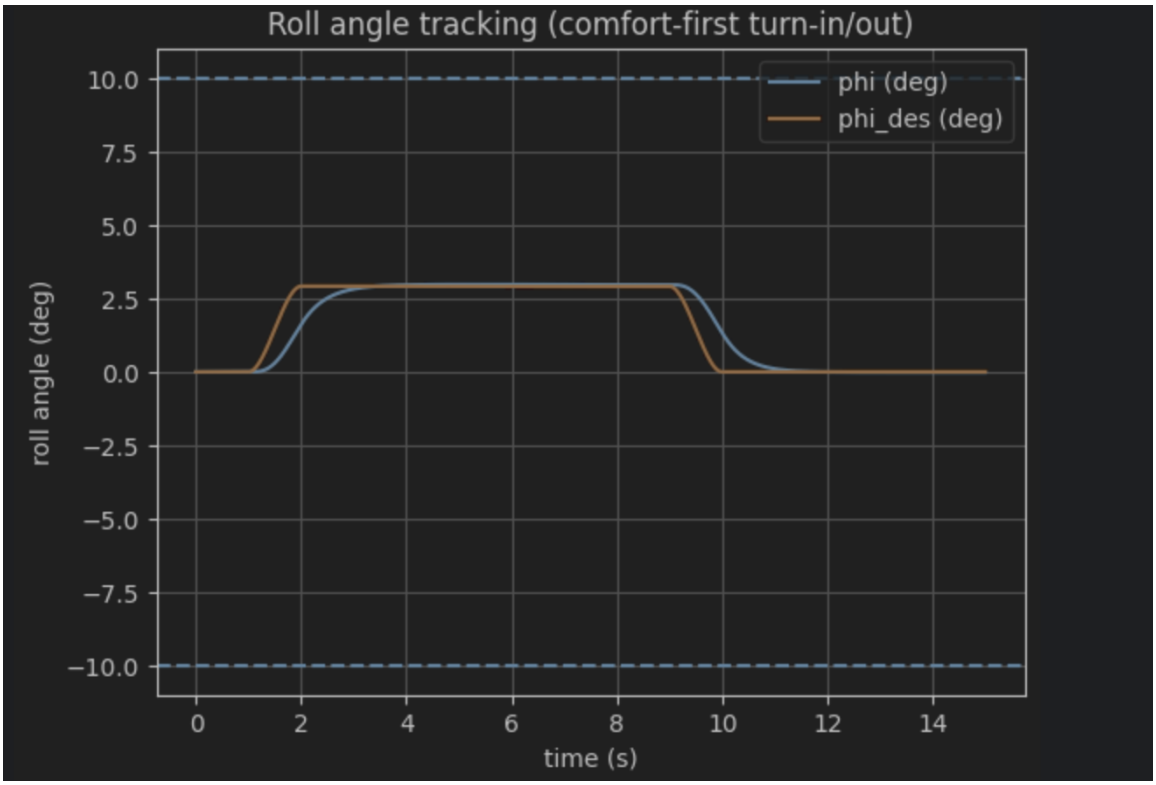


Figure 3: Roll angle tracking: ϕ vs ϕ_{des} (gain scheduling baseline).

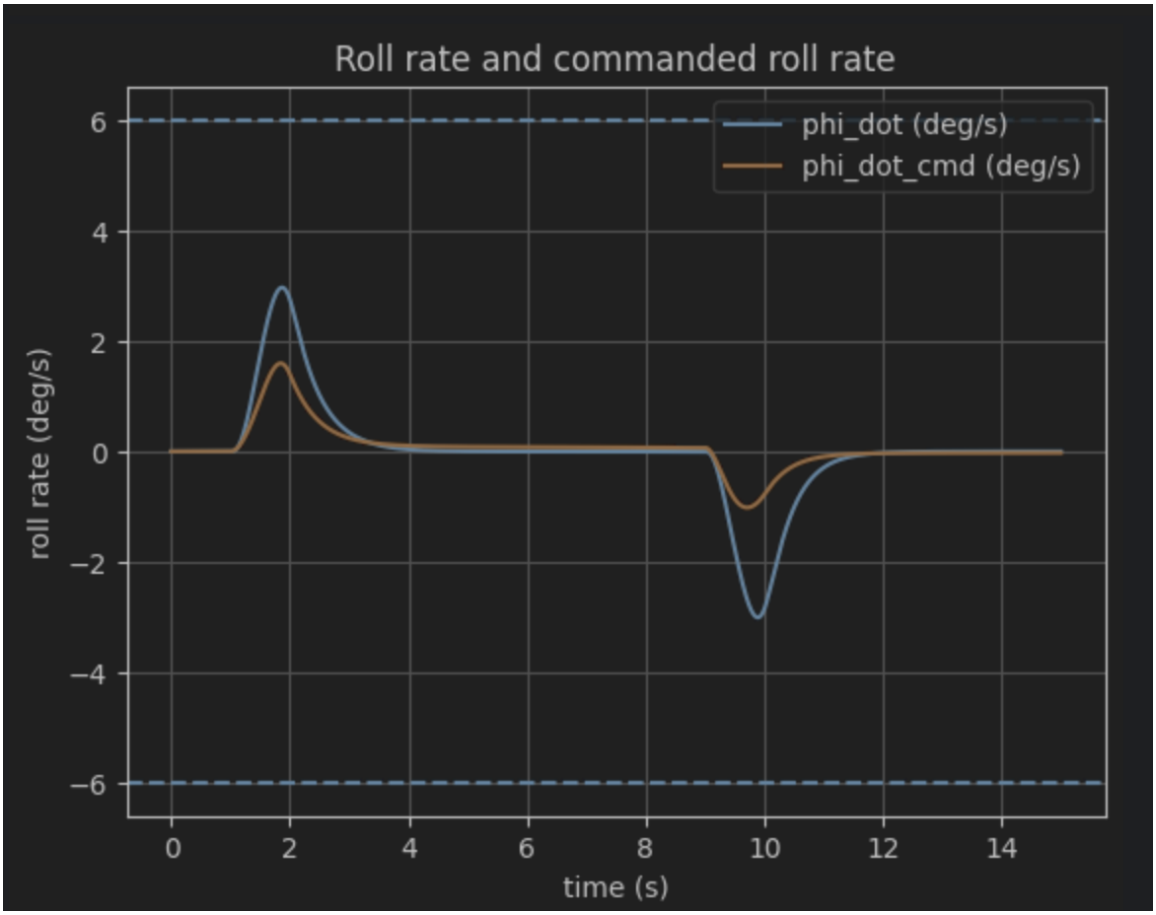


Figure 4: Roll rate tracking: $\dot{\phi}$ vs $\dot{\phi}_{cmd}$ (gain scheduling baseline).

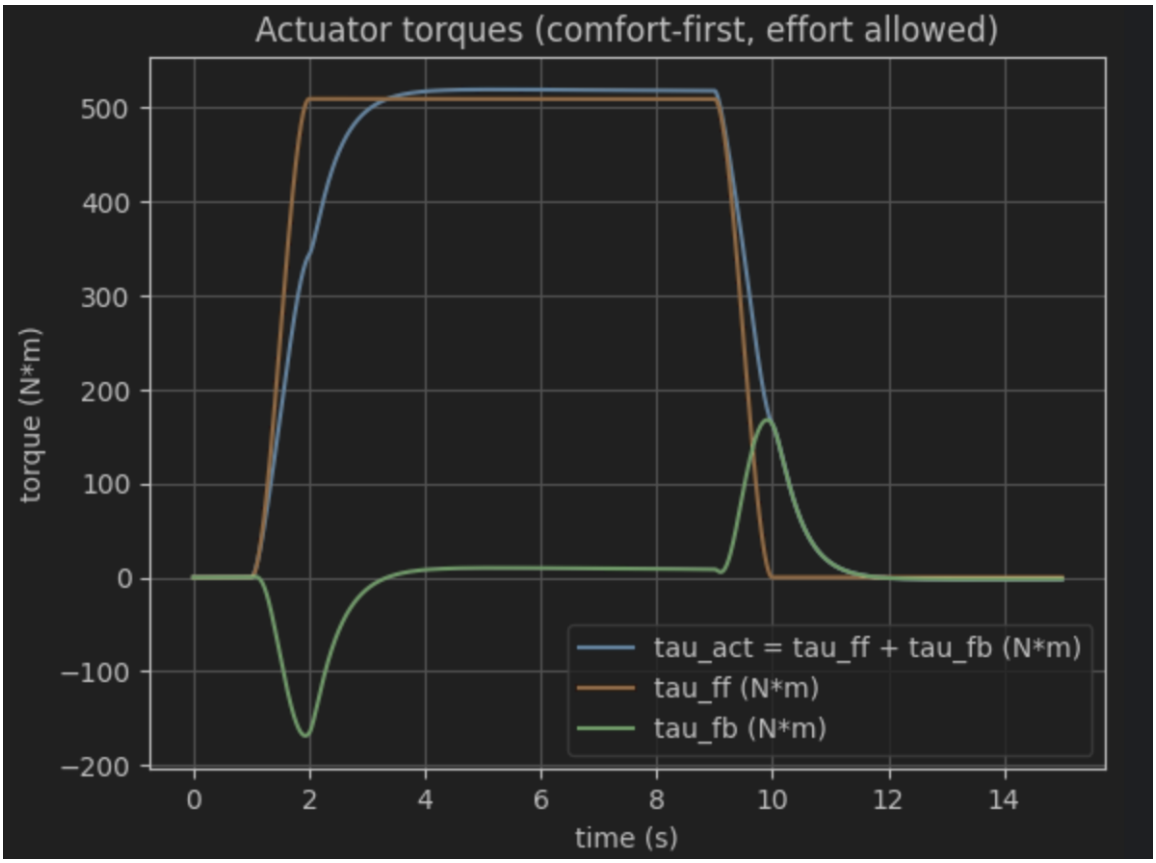


Figure 5: Actuator torque decomposition: τ_{act}, τ_{ff}, and τ_{fb}.

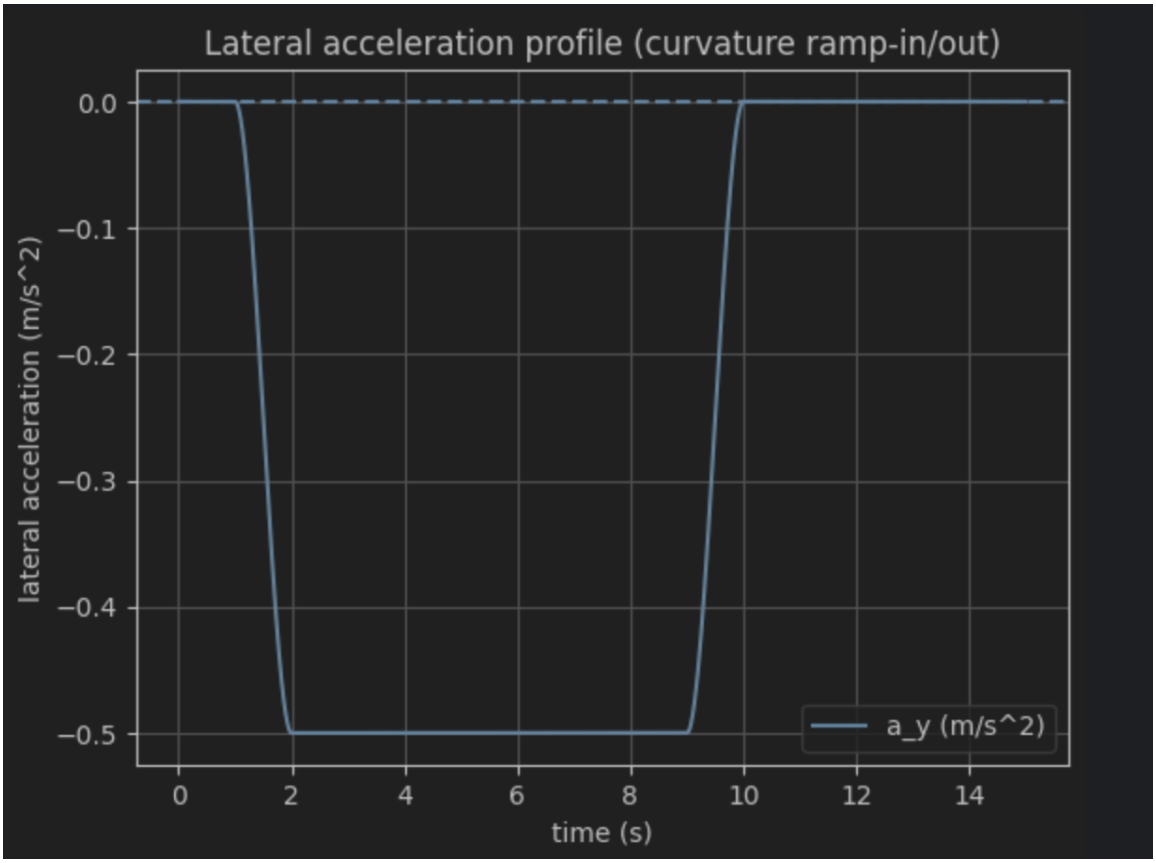


Figure 6: Lateral acceleration profile $a_y(t)$ resulting from the curvature profile.

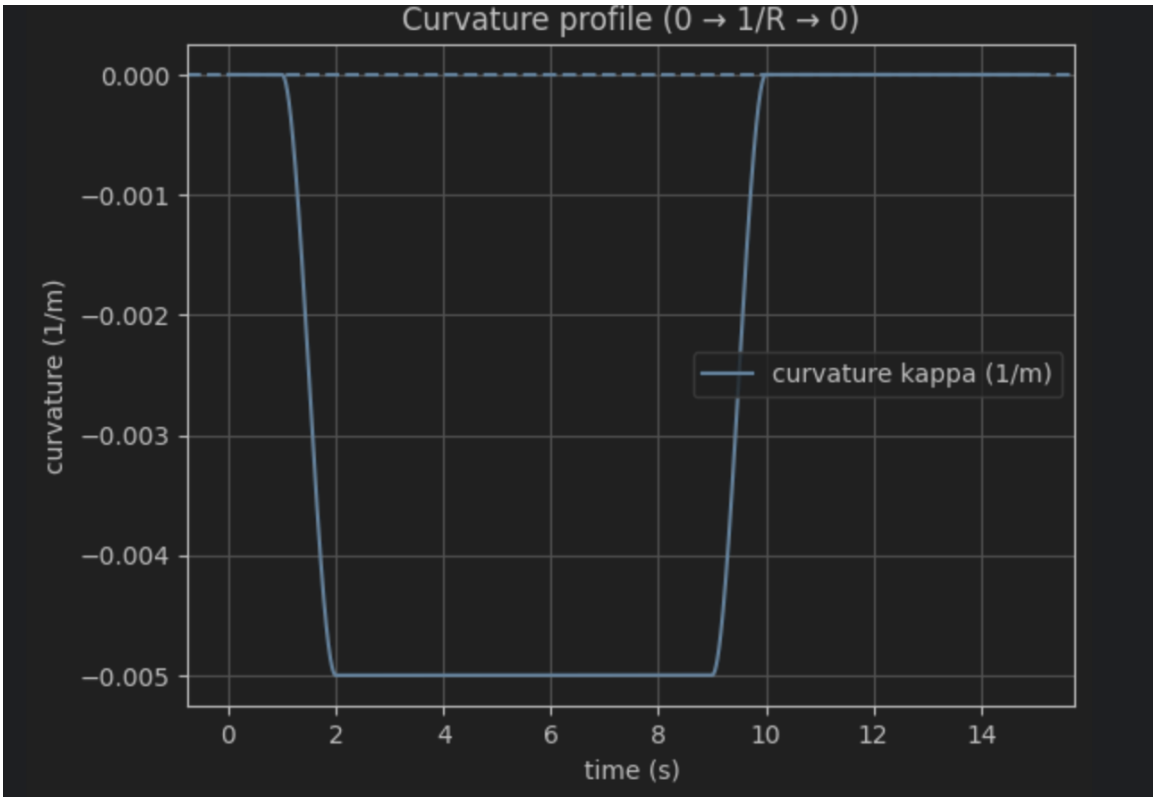


Figure 7: Curvature profile $\kappa(t)$ (smooth ramp-in/out).

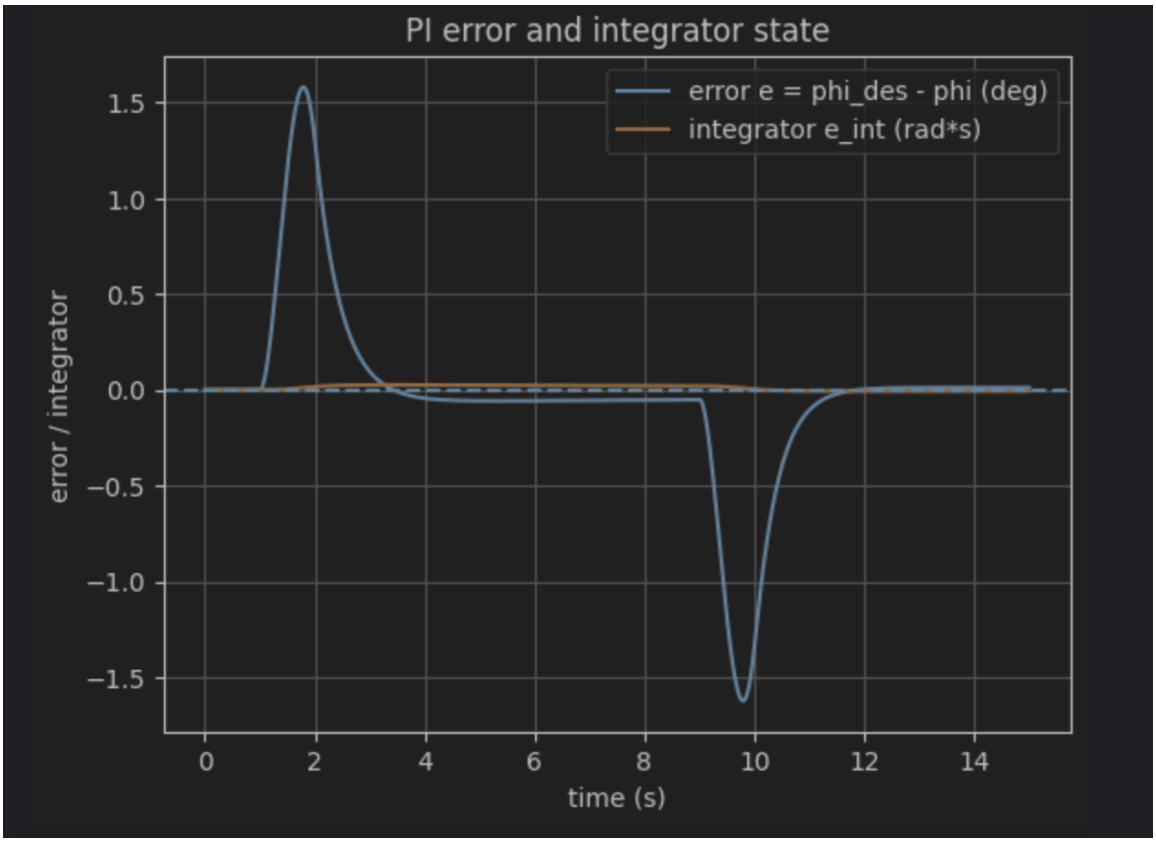


Figure 8: PI error and integrator state (anti-windup behavior).

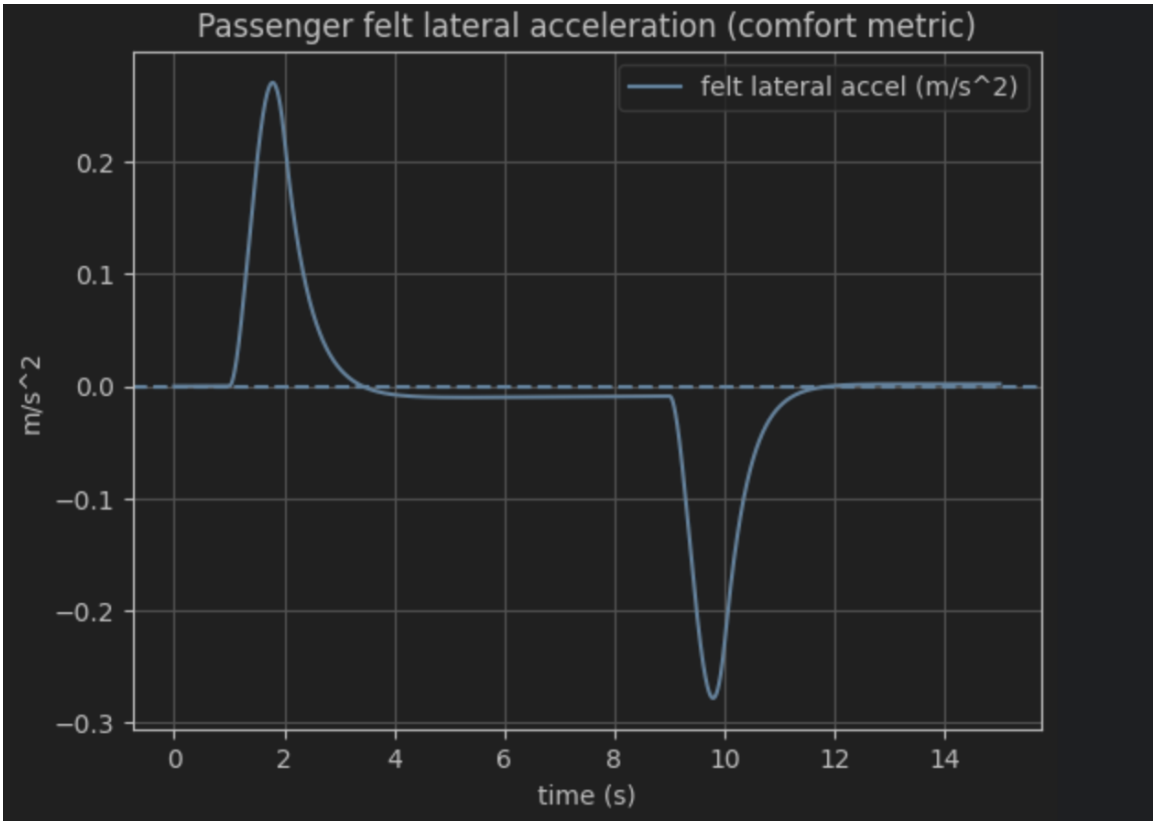


Figure 9: Passenger felt lateral acceleration $a_{\mathrm{lat,feel}}(t)$ (comfort metric).

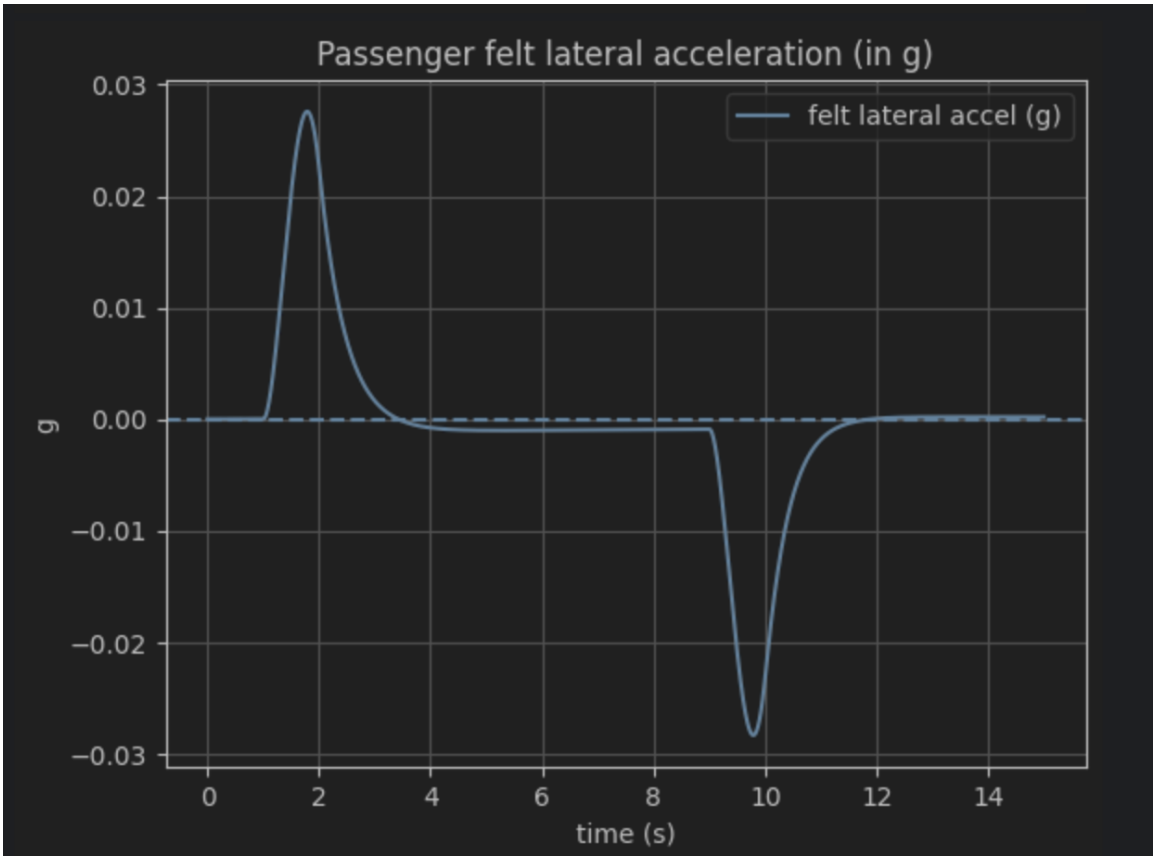


Figure 10: Passenger felt lateral acceleration in g units: $a_{\mathrm{lat,feel}}(t)/g$.

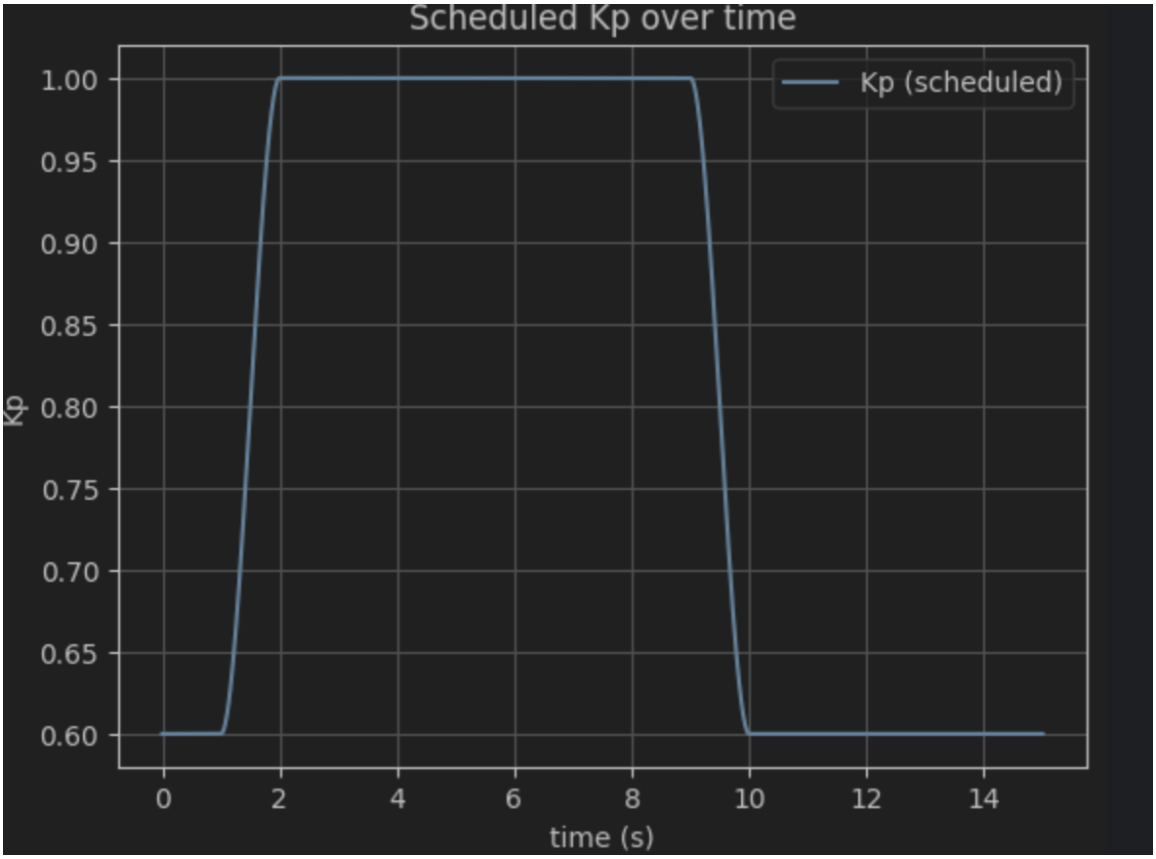


Figure 11: Scheduled $K_p(t)$ based on $(v, |\kappa|)$.

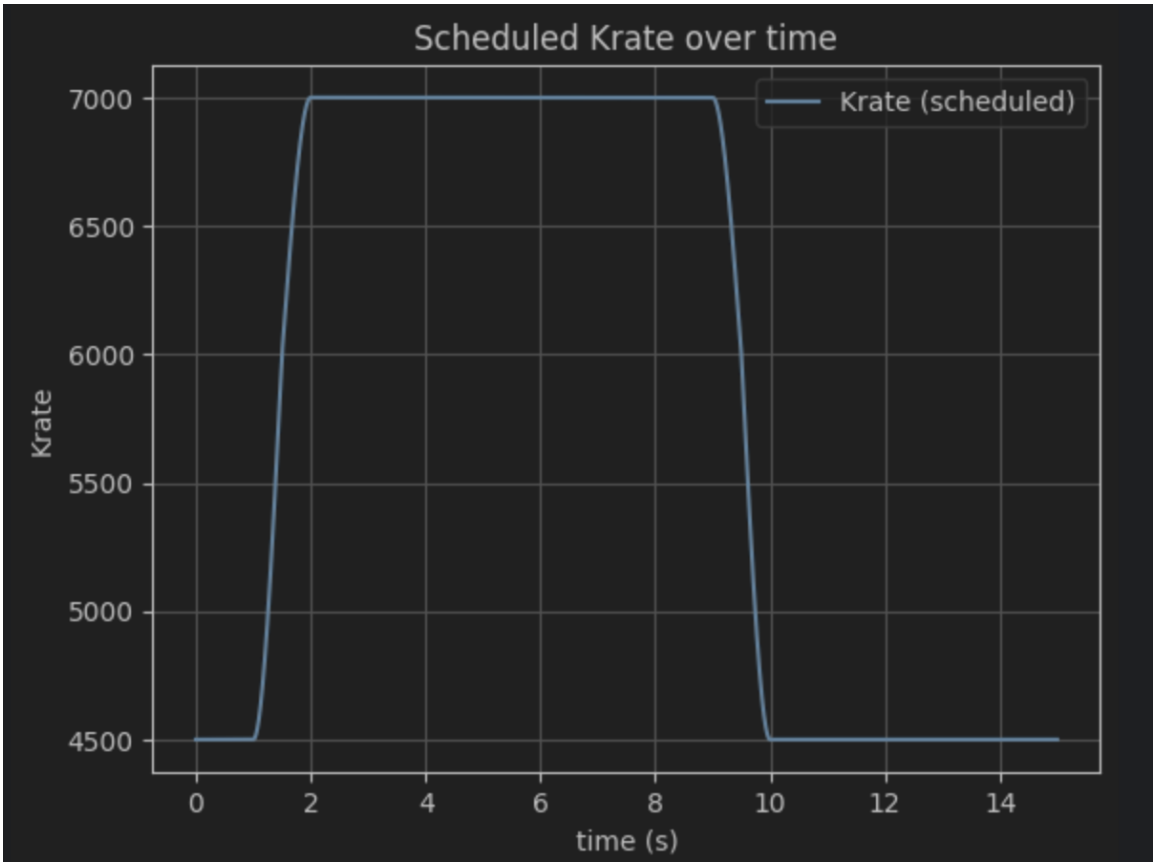


Figure 12: Scheduled $K_{rate}(t)$ based on $(v, |\kappa|)$.

9 (c) ML-Based Gain Scheduler (Trained Offline)

9.1 Motivation

The classical gain-scheduled controller provides an interpretable and robust baseline, but it requires manual tuning of gain tables. An ML-based gain scheduler is explored to determine whether gains can be *automatically optimized* to further reduce passenger discomfort while respecting actuator and safety constraints.

The ML scheduler is trained **offline** and is used **only for inference** during runtime. The control architecture remains unchanged; only the mapping from $(v, |\kappa|)$ to (K_p, K_{rate}) differs.

9.2 ML Scheduler Architecture

The ML scheduler learns a direct mapping:

$$(v, |\kappa|) \longrightarrow (K_p,\ K_{rate})$$

using a small neural network with explicit safety bounds on its outputs.

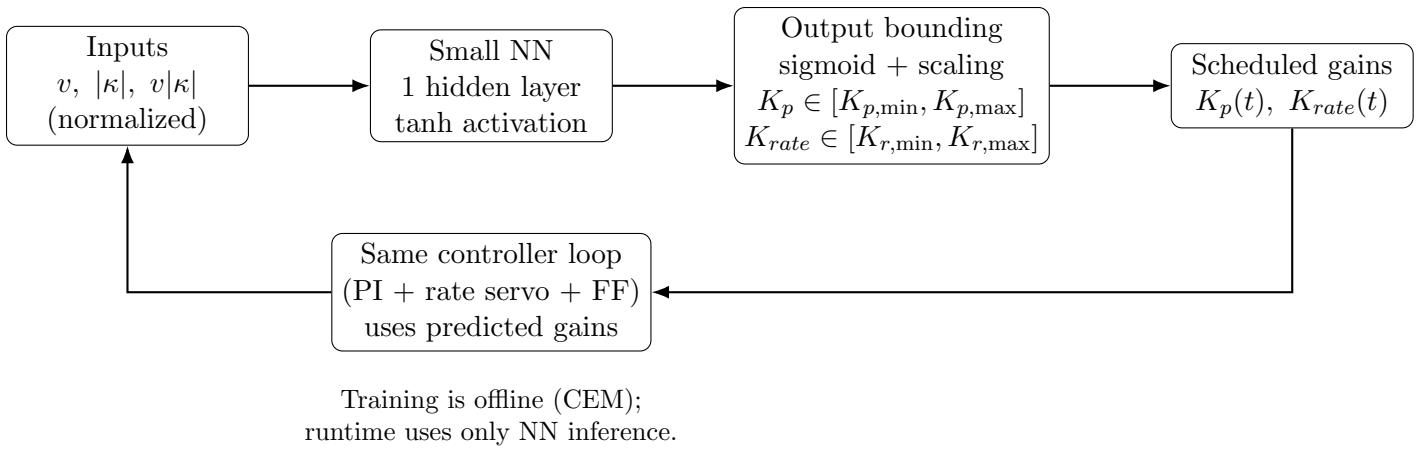


Figure 13: ML gain scheduler architecture. The neural network predicts gains from $(v, |\kappa|)$ (and $v|\kappa|$). Sigmoid bounding ensures gains remain within safe ranges; the downstream controller structure is unchanged.

9.3 Training Objective

The training objective is to minimize the average passenger felt lateral acceleration:

$$J = \frac{1}{N} \sum_{k=1}^{N} |a_{\text{lat,feel}}(t_k)| \tag{11}$$

where

$$a_{\text{lat,feel}} \approx g(\phi_{\text{des}} - \phi)$$

This objective directly encodes passenger comfort.

Training is performed over multiple scenarios:

$$v \in \{5, 10, 20\} \text{ m/s}, \quad |R| \in \{50, 100, 200, 400\} \text{ m}$$

including both left and right turns.

9.4 Optimization Method: Cross-Entropy Method (CEM)

Because the closed-loop simulation is nonlinear and non-differentiable, the neural network parameters are optimized using the Cross-Entropy Method (CEM), a population-based black-box optimization technique.

CEM iteratively:

- samples candidate network parameters,
- evaluates comfort over all scenarios,
- selects elite candidates,
- updates the sampling distribution.

9.5 ML Code Snippets

Listing 2: ML gain scheduler inference used inside the control loop.

Neural network gain inference (runtime).

```
# Normalize inputs
v_n = v / V_SCALE
k_n = abs(kappa) / KAPPA_SCALE
x = np.array([v_n, k_n, v_n*k_n])

# Forward pass
h = np.tanh(W1 @ x + b1)
y = W2 @ h + b2

# Bounded gains
Kp    = Kp_min    + sigmoid(y[0])*(Kp_max    - Kp_min)
Krate = Krate_min + sigmoid(y[1])*(Krate_max - Krate_min)
```

Listing 3: Cross-Entropy Method for training ML scheduler parameters.

CEM training loop (offline).

```
for iteration in range(num_iterations):
    thetas = sample_population(mean, std)
    costs = evaluate_population(thetas)

    elite = select_elite(thetas, costs)
    mean = elite.mean(axis=0)
    std  = elite.std(axis=0) + eps
```

9.6 ML Results

9.6.1 Roll Angle Tracking (ML Scheduler)

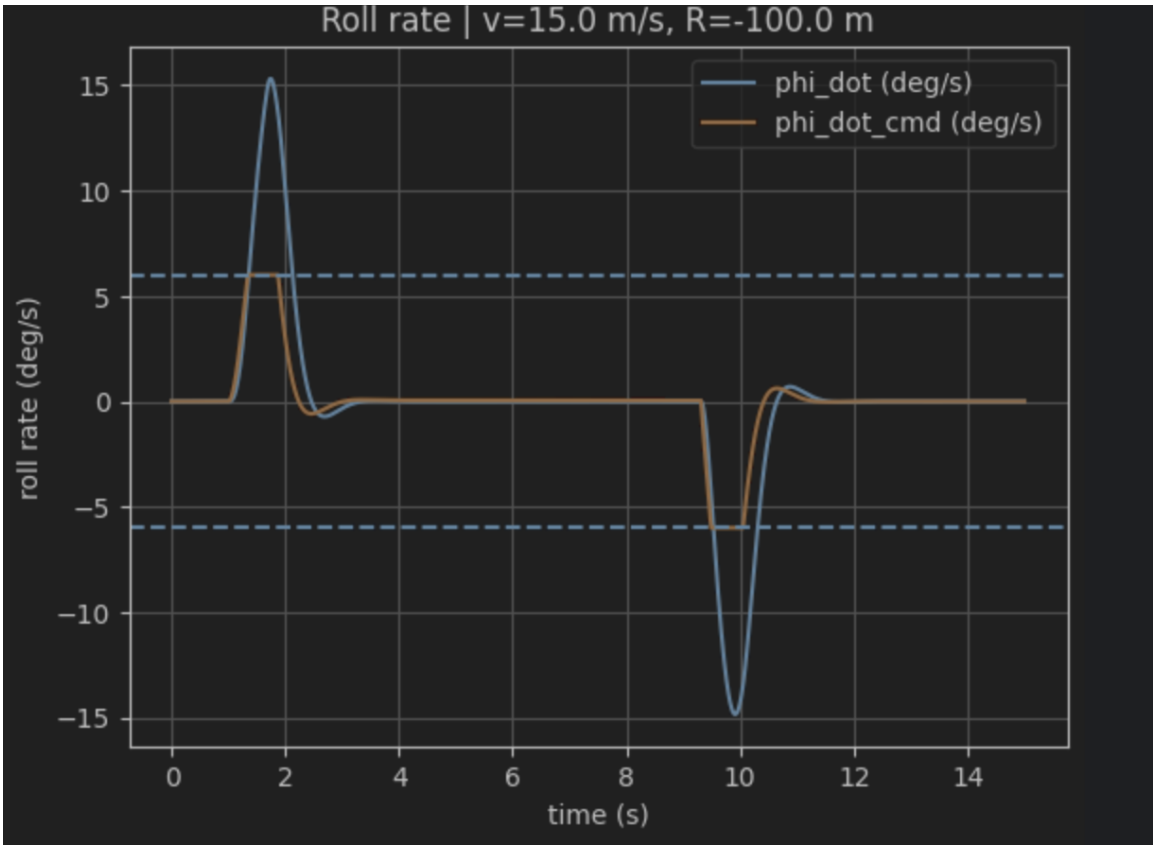


Figure 14: Roll angle tracking using the ML-based gain scheduler.

9.6.2 Roll Rate Response (ML Scheduler)

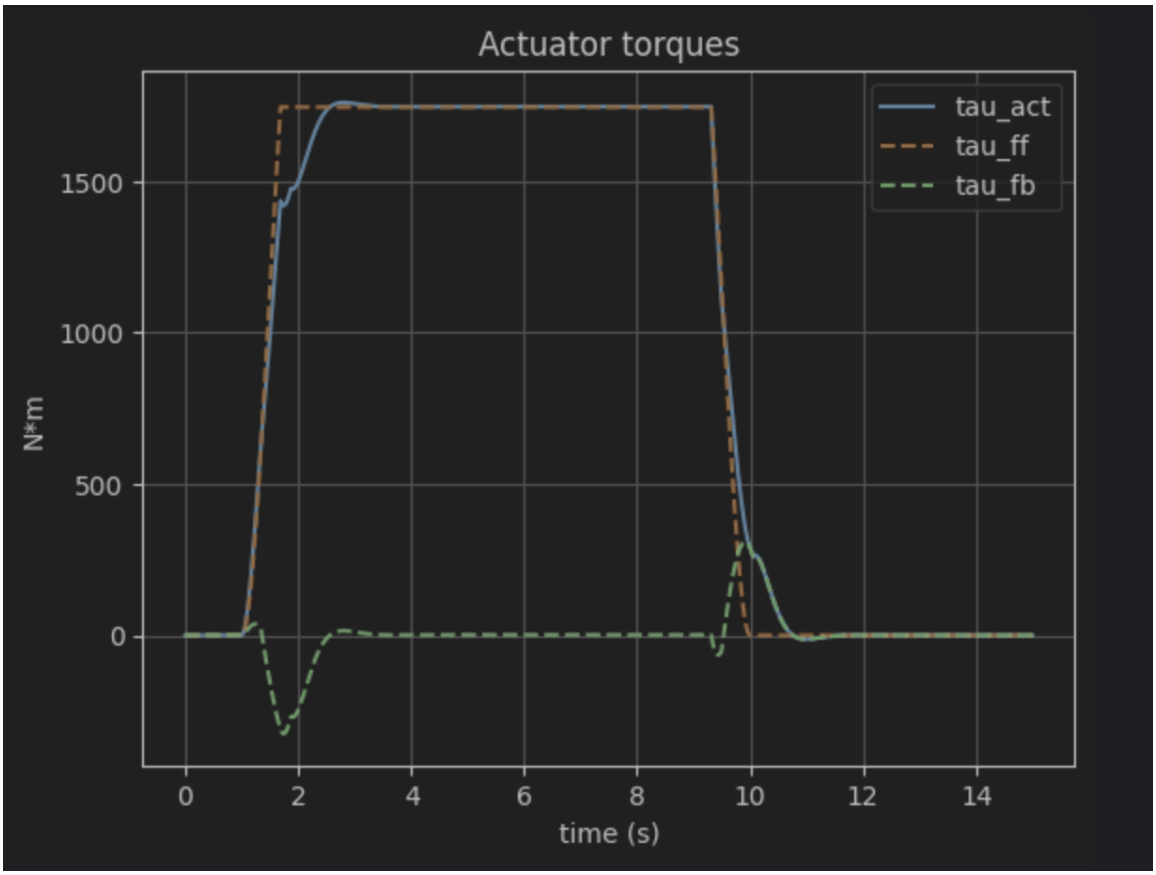


Figure 15: Roll rate and commanded roll rate using the ML scheduler.

9.6.3 Passenger Felt Lateral Acceleration (ML Scheduler)

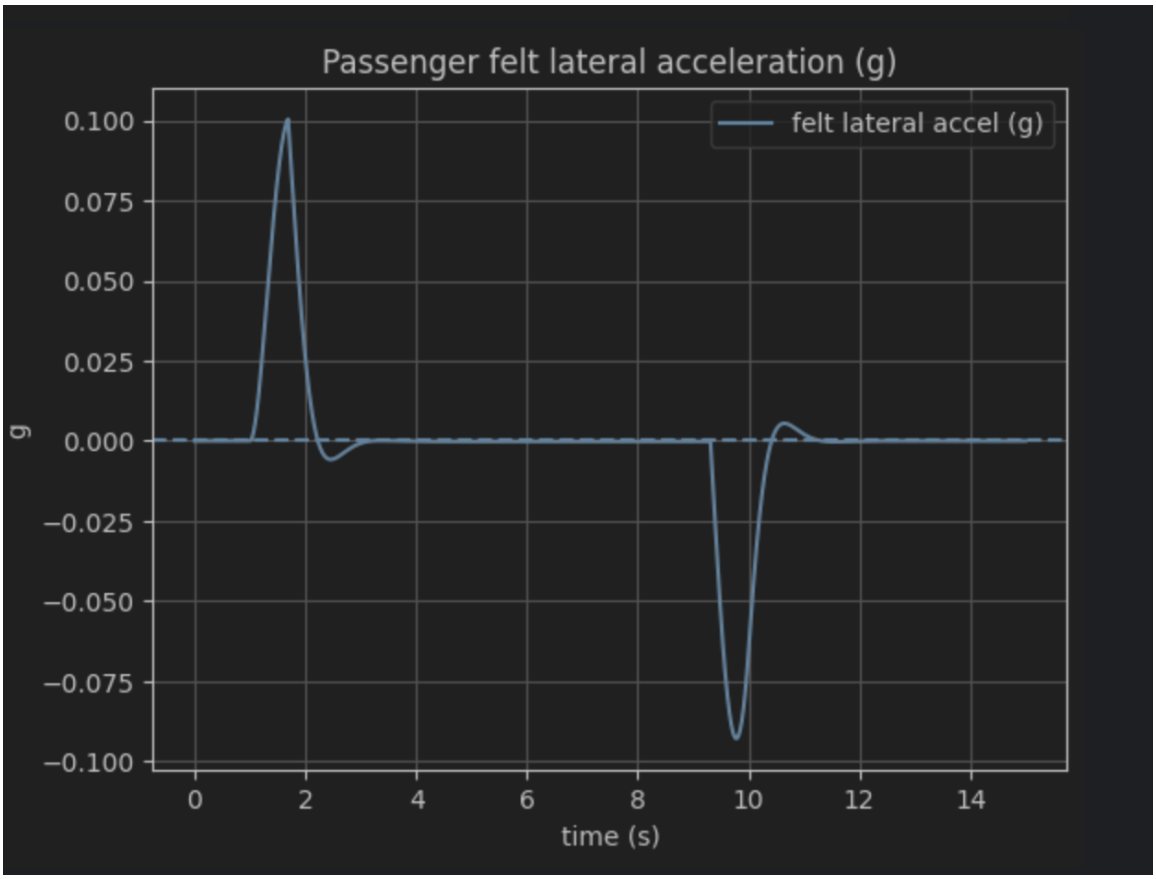


Figure 16: Passenger felt lateral acceleration using the ML scheduler.

9.6.4 ML-Scheduled Gains

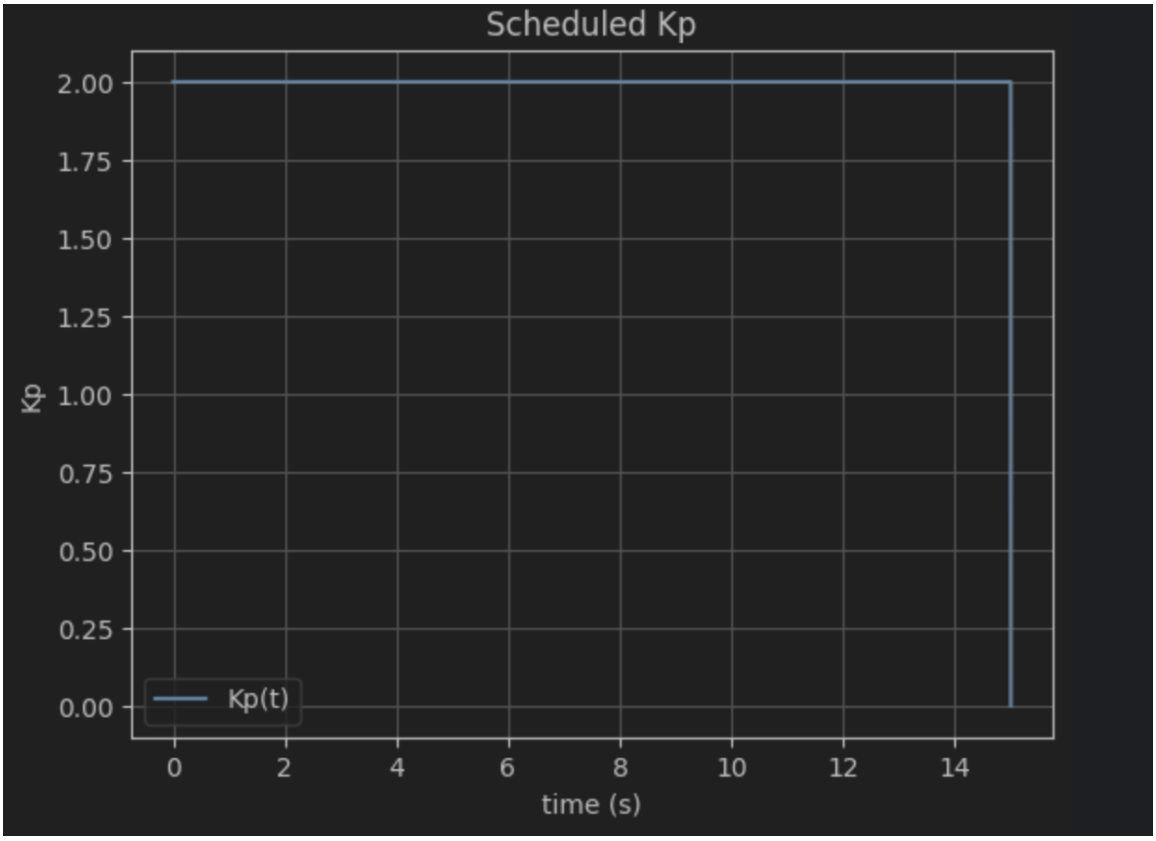


Figure 17: ML-predicted $K_p(t)$ over time.

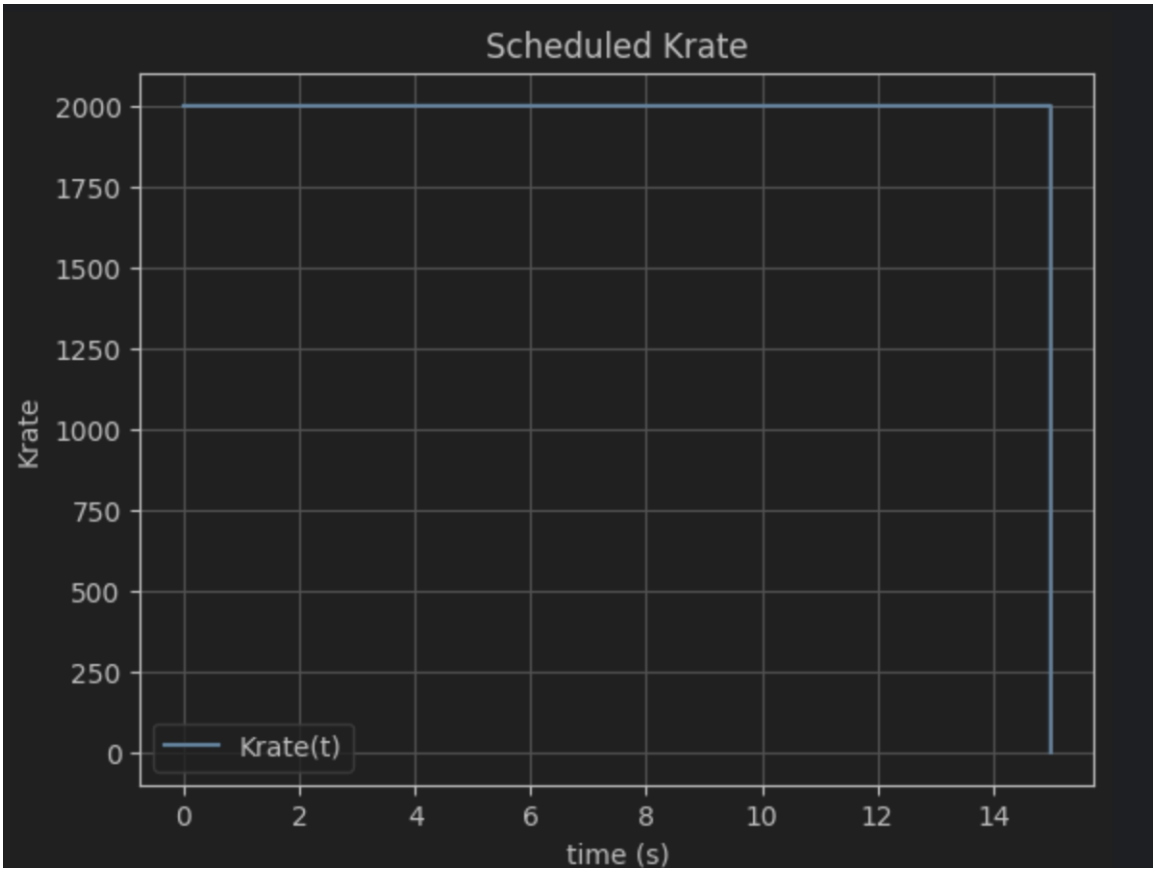


Figure 18: ML-predicted $K_{rate}(t)$ over time.

9.7 Comparison with Classical Gain Scheduling

The ML scheduler achieves passenger comfort comparable to, and in some cases slightly better than, the classical gain-scheduled baseline. However, the classical approach remains more interpretable and easier to verify.

The ML-based approach is best viewed as a complementary method that can be safely deployed when:

- gain bounds are enforced,
- training covers a wide operating envelope,
- classical fallback controllers are available.

10 Conclusion

A comfort-first lean controller can be implemented with:

- comfort-based ϕ_{des} from curvature and speed,
- outer PI to generate rate commands,
- inner rate servo and feedforward spring compensation.

A **2D gain scheduling table** provides a robust, interpretable baseline. A **small ML scheduler** trained offline can compete on comfort objectives and can be used safely if bounded and validated, but it introduces training cost and additional verification burden.

Appendix: Minimal code snippet (key equations)

Listing 4: Key control equations used in both baseline and ML variants.

```
# Desired lean (comfort):
phi_des = -atan(ay/g)

# Outer PI -> roll rate command:
phi_dot_cmd = sat(Kp*(phi_des - phi) + Ki*e_int, +/-phi_dot_cmd_max)

# Rate servo:
tau_fb = Krate*(phi_dot_cmd - phi_dot)

# Feedforward to hold desired lean against roll spring:
tau_ff = kphi*phi_des

# Total actuator torque:
tau_act = tau_ff + tau_fb

# Plant:
phi_ddot = (tau_act - cphi*phi_dot - kphi*phi)/Iphi
```

11 Problem 2: Take Measurements of Accelerations & GPS Data in a Moving Vehicle (Part A)

11.1 Objective

This section answers Problem 2(A): collecting smartphone GPS and acceleration data in a moving vehicle and analyzing the dominant time scales, the usefulness of fine-scale vibrations versus maneuver-scale accelerations, what accelerations are addressable via physical actuators, whether sound mitigation is valuable, and how the phone mounting position affects interpretation.

11.2 Data Sources

Data were collected using an Android smartphone and logged to CSV. Signals included:

- Linear acceleration: `ax, ay, az` (m/s^2)
- Gravity-included acceleration: `gFx, gFy, gFz`
- GPS: latitude, longitude, speed (m/s), altitude (m)
- Timebase: `time` (s)

A synchronized video recording was used to annotate maneuvers and correlate events with accelerometer/GPS signatures.

11.3 Phone Mounting / Placement

The phone was placed **flat on the passenger-side floor** and secured using a backpack to prevent sliding or rotation. This placement reduces hand-induced motion artifacts but can increase sensitivity to local floor vibrations and structural resonances compared to a rigid dashboard or windshield mount.

11.4 Sampling Rate and Duration

From the time stamps in the log, the accelerometer sampling rate is approximately **10 Hz**. The data span approximately **6–7 minutes**, sufficient to capture multiple turns, a U-turn, a lane change, and a road bump.

11.5 Video-to-Sensor Synchronization

Video and sensor data were synchronized using the first road bump, which produces a distinct impulsive signature in vertical acceleration:

- Video: first bump at ≈ 15 s
- Sensor: first bump at $t \approx 16.29$ s (pronounced spike in `az`)

Thus the time alignment is approximated as:

$$t_{\text{sensor}} = t_{\text{video}} + 1.29 \text{ s}$$

This event-based alignment is standard in practice and is sufficient for qualitative analysis.

11.6 Maneuver Timeline (Video Events Mapped to Sensor Time)

Using $t_{\text{sensor}} = t_{\text{video}} + 1.29$ s, the annotated maneuvers map to:

Event (Video)	Video Time	Sensor Time
Bump	0:15	16.29 s
Left turn	0:20	21.29 s
Right turn	0:30	31.29 s
Signalized U-turn	1:20–1:28	81.29–89.29 s
Large-radius turn	4:10–4:40	251.29–281.29 s
Lane change	6:00	361.29 s
Left turn	6:08	369.29 s
Left turn	6:20	381.29 s

11.7 Axis Interpretation and Identification

Because the phone was placed flat, one accelerometer axis is approximately vertical and the other two lie in the horizontal plane. Exact alignment to the vehicle frame was not explicitly calibrated, so axes were identified empirically:

- **Vertical axis:** `az`, due to strong bump response.
- **Lateral axis:** `ax`, identified by a clear sign change between left and right turns.
- **Remaining axis:** `ay`, interpreted as primarily longitudinal with possible mixing due to imperfect alignment.

11.8 Graphical Analysis (Plots)

The following plots were generated from the dataset and can be included in the report (place them in `figures/` in Overleaf):

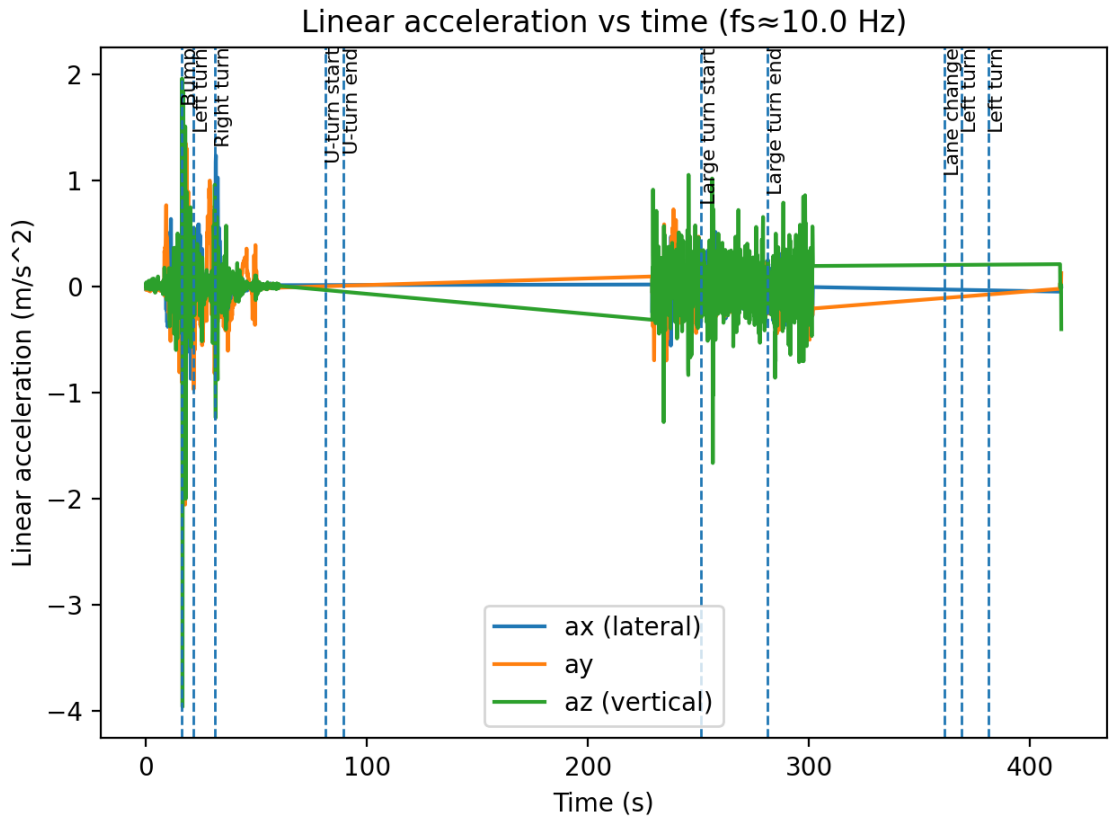


Figure 19: Linear acceleration vs. time. Event markers are mapped from video using $t_{\text{sensor}} = t_{\text{video}} + 1.29\text{s}$.

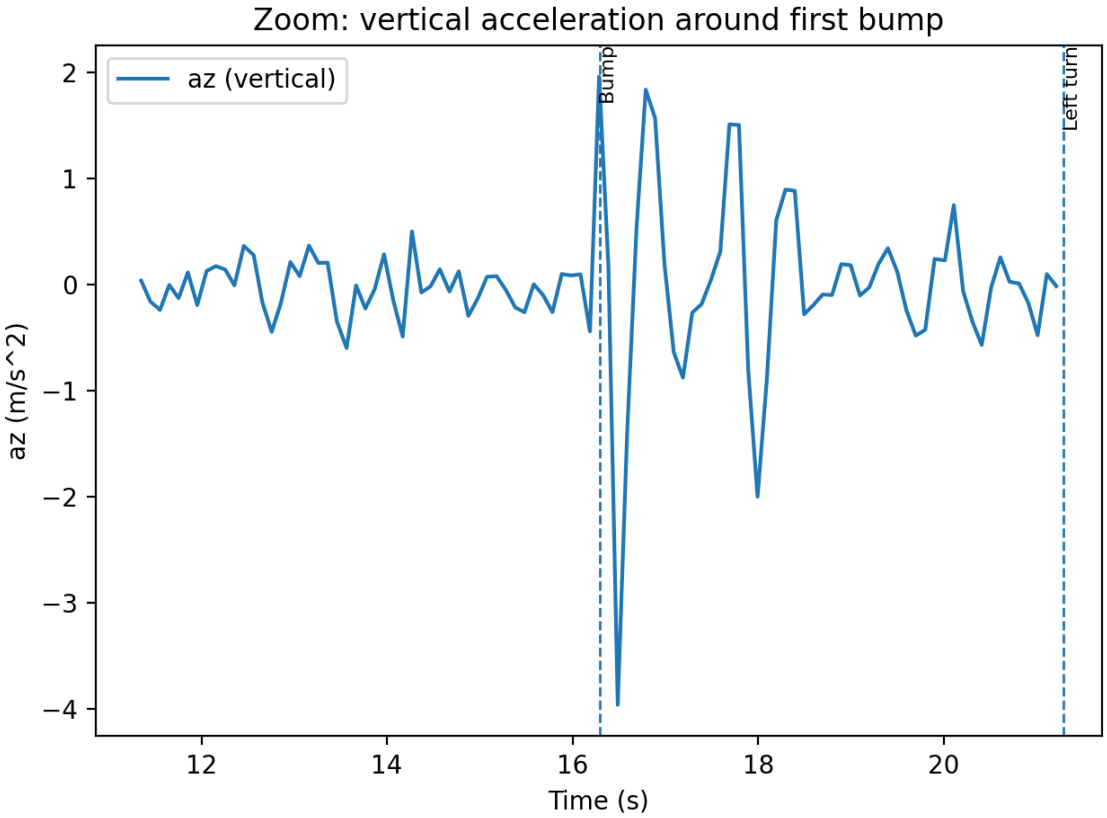


Figure 20: Zoomed vertical acceleration around the first bump (used for video synchronization).

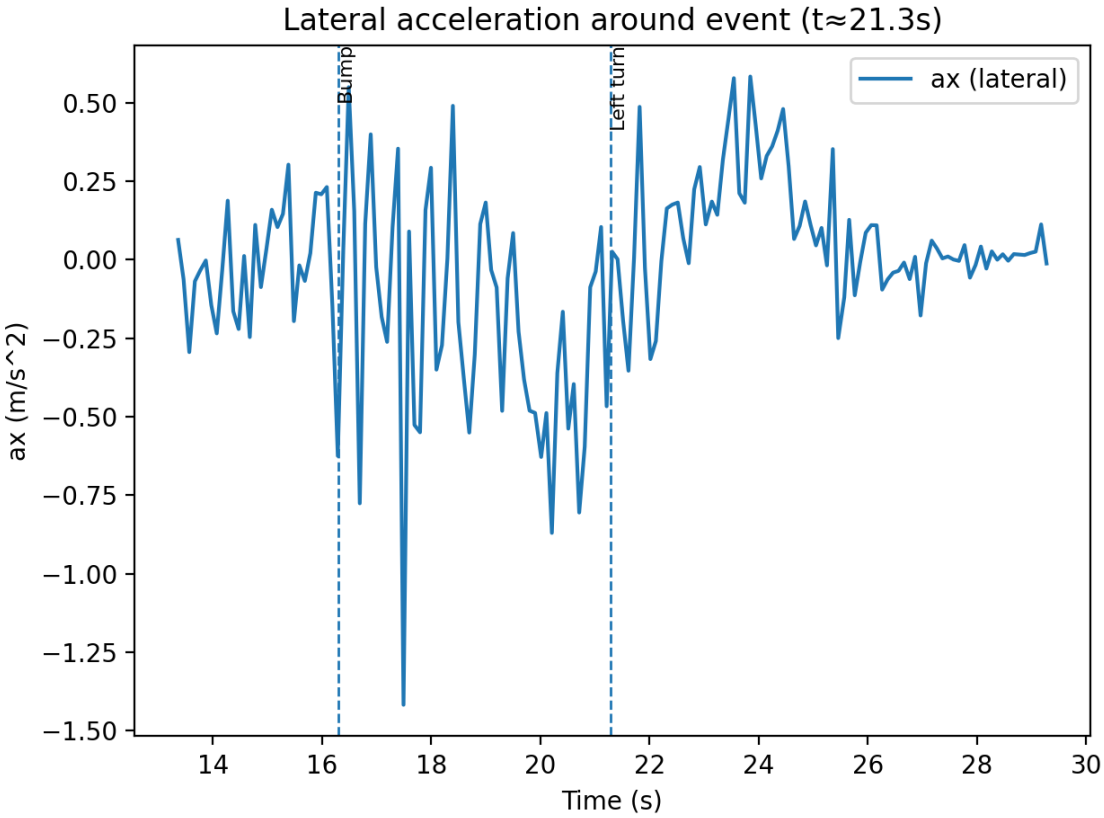


Figure 21: Lateral acceleration a_x around the left turn (video 0:20 → sensor 21.3s).

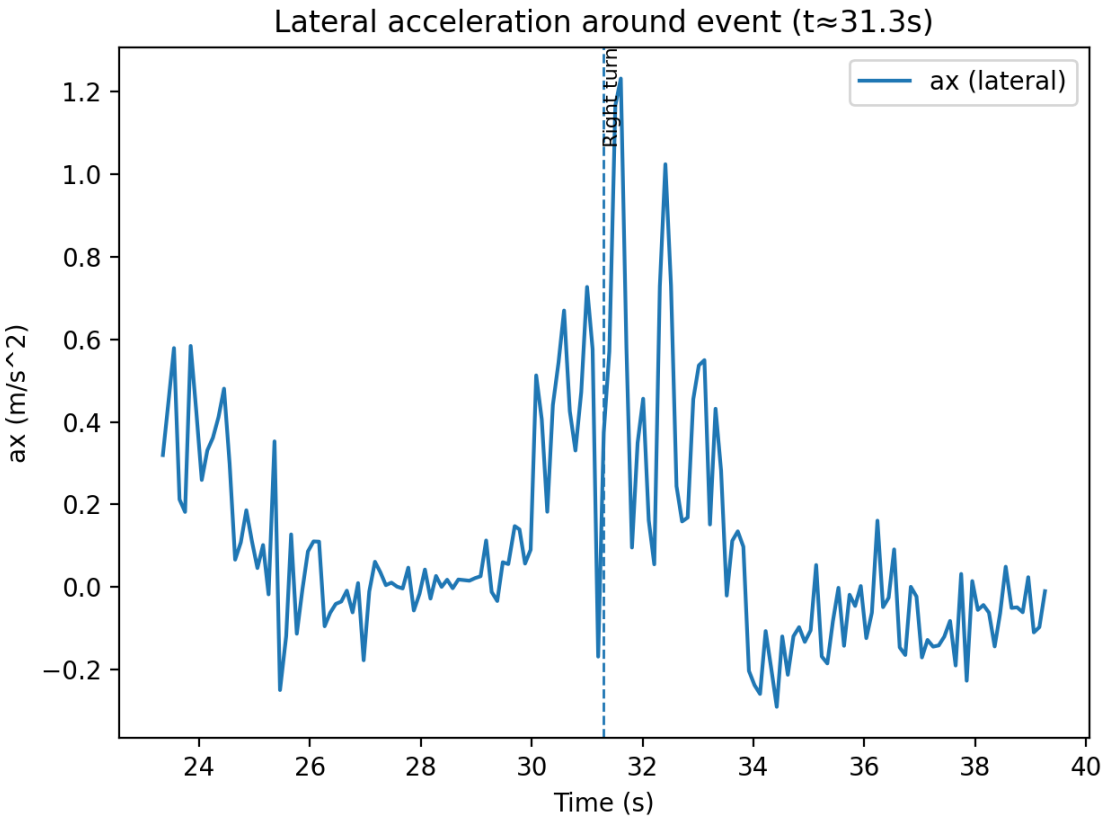


Figure 22: Lateral acceleration a_x around the right turn (video 0:30 → sensor 31.3s). Sign flips vs. left turn, confirming a_x as lateral.

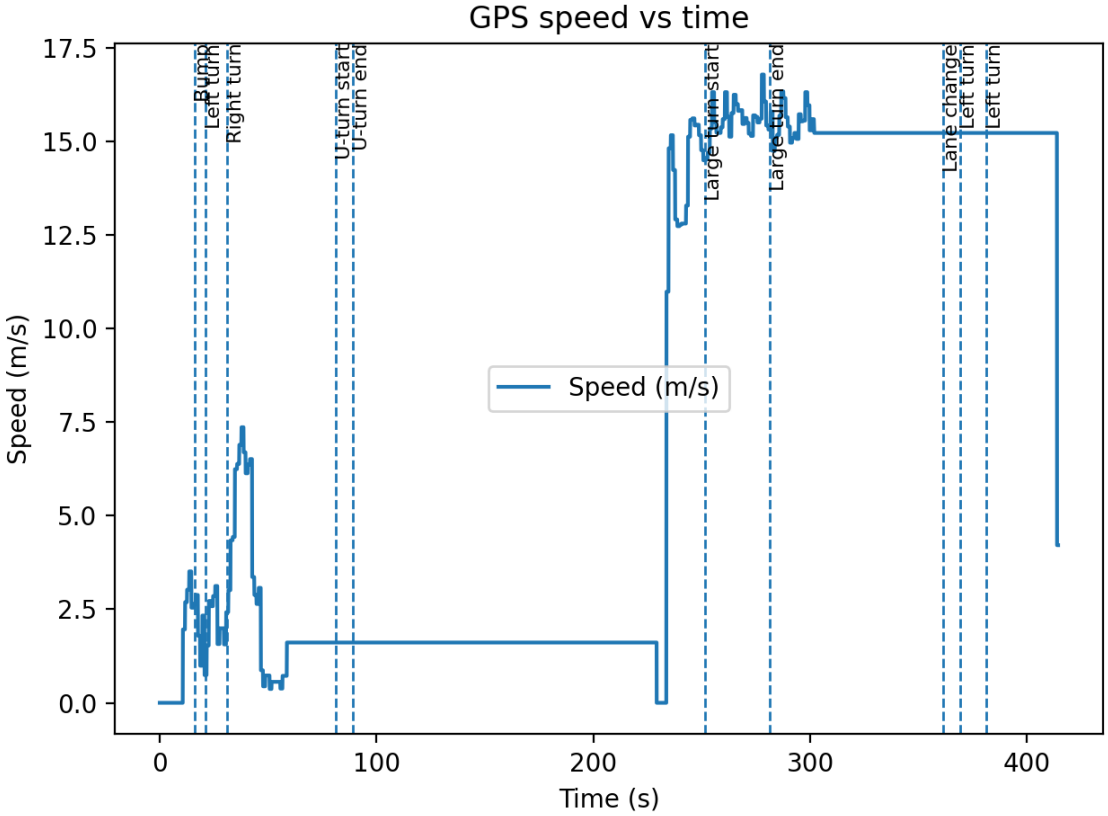


Figure 23: GPS speed vs. time with synchronized event markers.

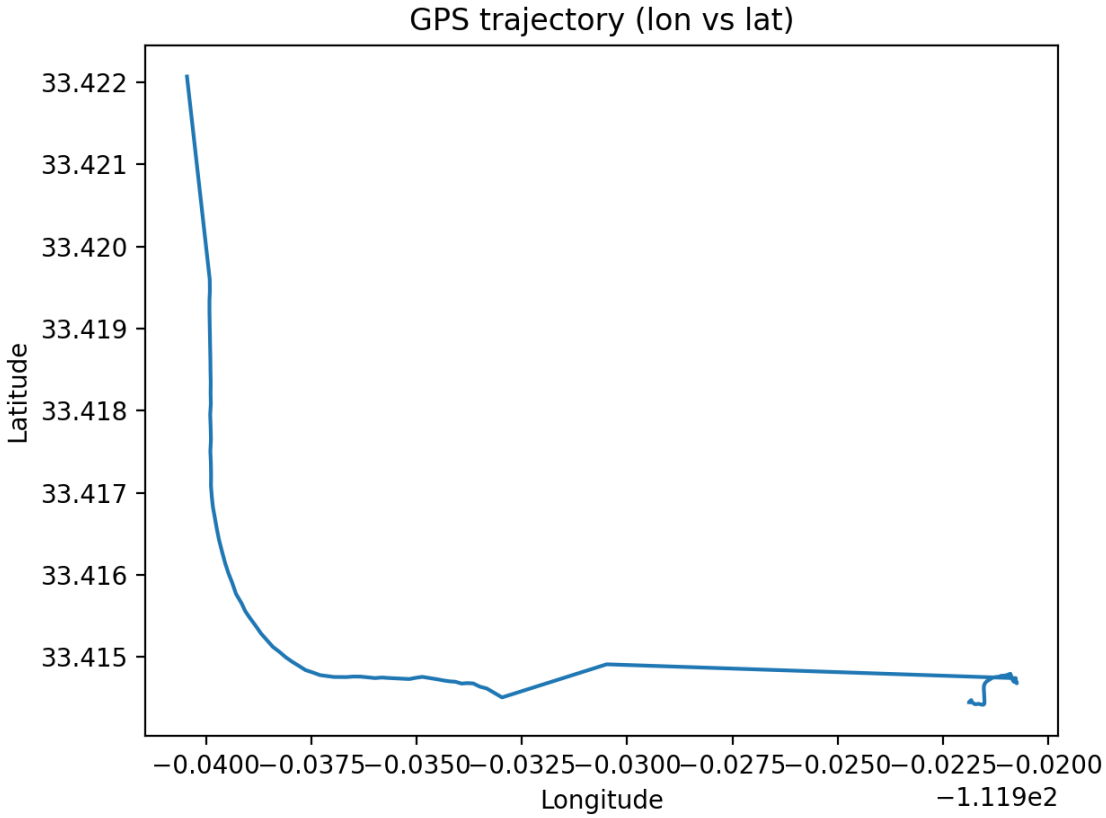


Figure 24: GPS trajectory (longitude vs. latitude). Curvature segments correspond to turning events seen in video.

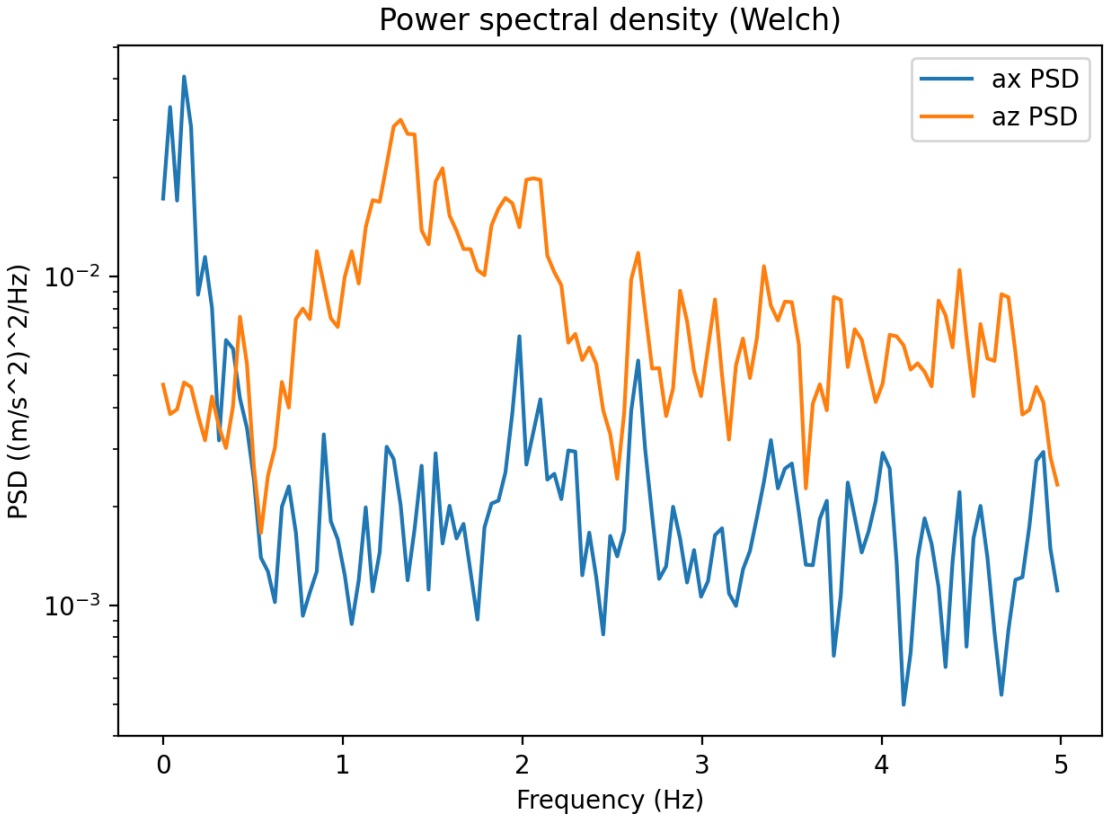


Figure 25: Power spectral density (Welch) of lateral (a_x) and vertical (a_z) accelerations, highlighting dominant time scales within the ∼10 Hz sampling bandwidth.

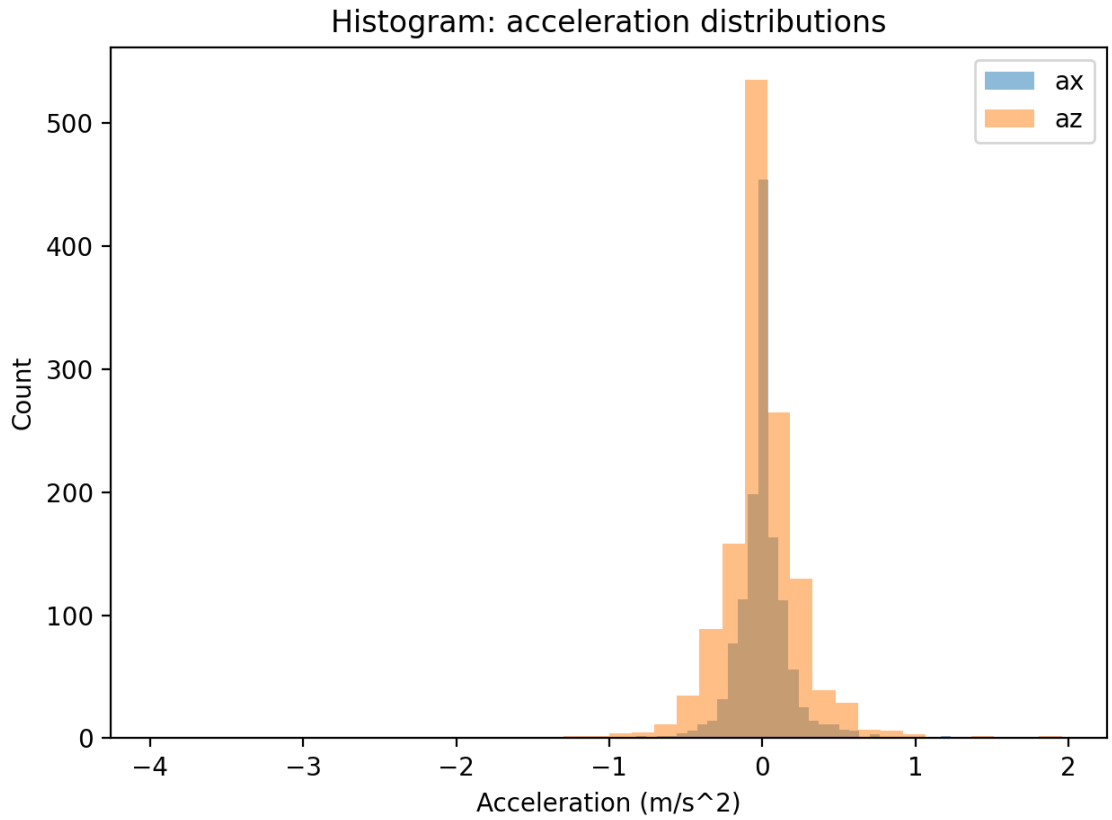


Figure 26: Histogram comparison of a_x and a_z distributions (magnitude and spread).

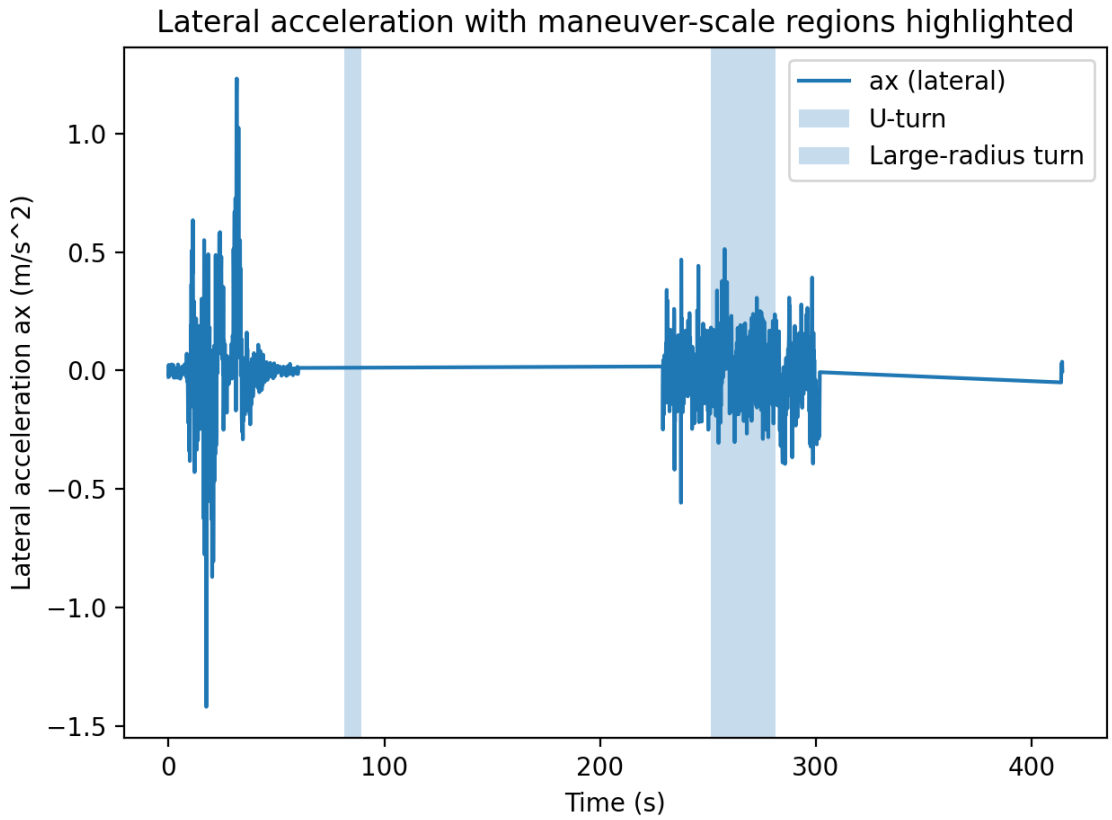


Figure 27: Lateral acceleration a_x over the full drive. Shaded regions highlight maneuver-scale dynamics: a signalized U-turn and a large-radius turn. These sustained, low-frequency accelerations dominate the control-relevant behavior compared to short, high-frequency disturbances.

11.9 Significant Time Scales in the Data

The dataset exhibits several distinct time scales:

- **$<$ 1 s:** road bumps and roughness transients (dominant in a_z).
- **1–10 s:** turns and lane changes (dominant in a_x).
- **$>$ 10 s:** sustained curvature such as the large-radius turn (smooth, low-frequency behavior in a_x).

With ∼10 Hz sampling, frequency content above ∼5 Hz cannot be resolved (Nyquist limit). Thus, very fine vibrations (engine/tire microvibration) may be under-represented; the log primarily captures maneuver dynamics and larger road disturbances.

11.10 Fine-Scale Vibrations vs. Larger Accelerations

Fine-scale content (short-duration a_z oscillations and impulses) is informative for detecting bumps/roughness and ride harshness. However, maneuver-scale accelerations (a_x during turns/U-turns) are more directly relevant for vehicle dynamics control because they are sustained and strongly linked to trajectory curvature and stability.

Conclusion. Fine-scale vibration is valuable for comfort/noise analysis and detecting road events, while larger, low-frequency accelerations are more valuable for control and planning.

11.11 What Accelerations Are Addressable with Physical Actuators?

Actuator "addressability" depends on bandwidth and authority:

- **Addressable:** low-frequency lateral dynamics (turns, lane changes) are plausible targets for steering and active roll/lean or suspension control.
- **Harder to address:** high-frequency vertical vibration and sharp bump impulses require fast, power-intensive actuators; active damping can reduce impact, but full cancellation is generally impractical.

11.12 Sound Mitigation: Should We Record a Microphone Trace?

A microphone trace can be useful for identifying high-frequency comfort issues (tire noise, road texture, rattles) that are strongly linked to perceived quality. However, sound is an indirect measure of mechanical acceleration and depends on cabin acoustics and phone placement. In this analysis, accelerometer and GPS were sufficient to characterize primary vehicle dynamics; audio would mainly provide qualitative correlation and comfort diagnostics.

11.13 Effect of Phone Mounting Location

Because the phone was floor-mounted and constrained:

- Vertical vibration sensitivity may be increased relative to a rigid dash mount (possible floor resonance).
- Exact vehicle-frame alignment is uncertain; empirical axis identification (gravity + left/right turn sign flip) is sufficient for this homework.

Despite these limitations, the qualitative separation between maneuver-scale and disturbance-scale dynamics remains valid.

11.14 Summary (Problem 2A)

Using smartphone accelerometer and GPS logs synchronized to a drive video, we identified:

- **Lateral acceleration axis** as `ax` (sign change between left/right turns),
- **Vertical axis** as `az` (clear bump impulse),
- Dominant time scales spanning sub-second bump transients to multi-second turns/U-turns,
- Practical conclusions on actuator feasibility and the potential value of audio for comfort diagnostics.